



FINAL PROJECT PRESENTATION

- Single-Source Shortest Paths with
- Negative Real Weights in $O(mn^8/9)$ Time
-

Hussain Umer (08040) & Hussain Mustansir (08436)



MOTIVATION

- Problem Importance:
 - Many real-world graphs (finance, physics, networking) have negative weights.
 - Example: Currency arbitrage, where negative cycles mean profit opportunities.
- Why is this hard?
 - Dijkstra's fails with negative weights.
 - Bellman-Ford ($O(mn)$) is slow for large graphs.
- Our Goal:
 - First asymptotic improvement ($O(mn^{8/9})$) for exact SSSP with negative weights.

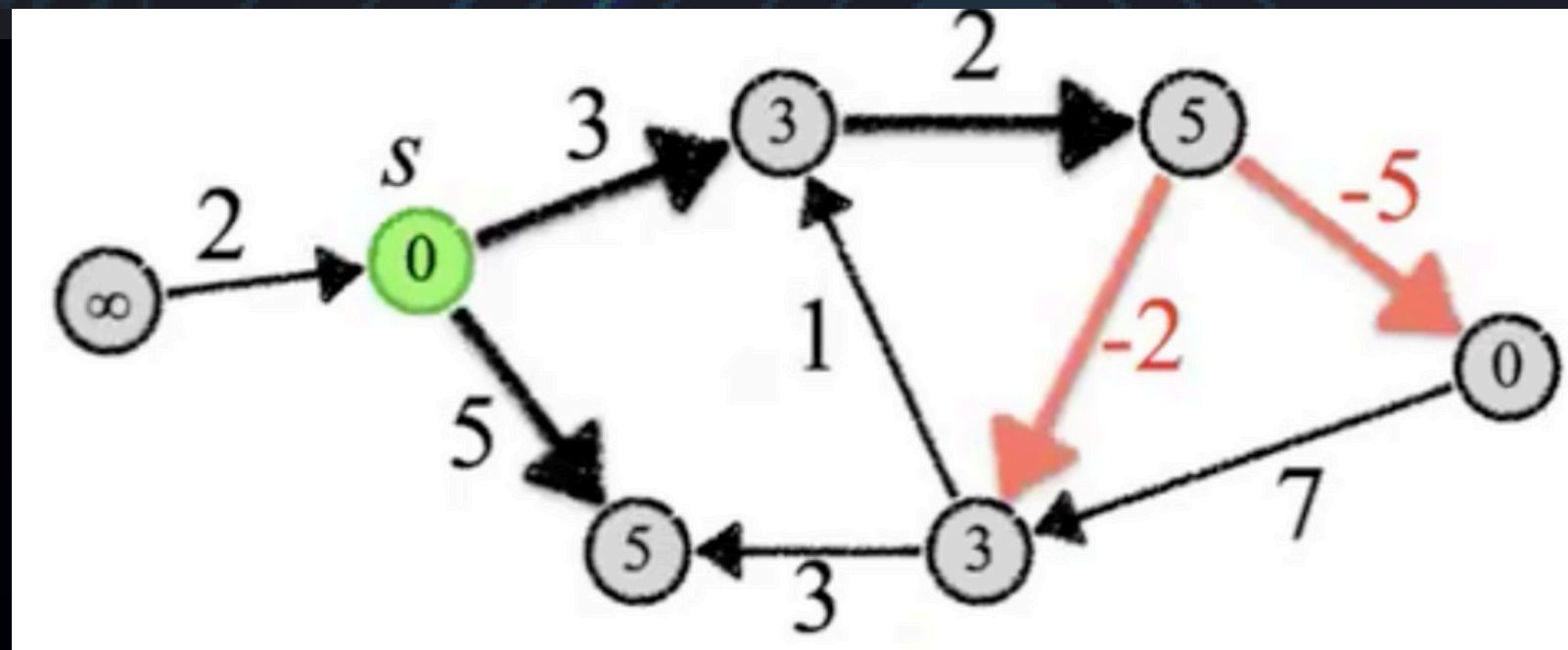
PROBLEM STATEMENT

Input:

A directed graph $G = (V, E, w)$ with real weights (positive & negative) and no negative cycles.

Output:

Shortest path from a source node s to all other nodes $v \in V$.



RELATED WORK & LIMITATIONS

Bellman-Ford (1958)
 $O(mn)$ time, works but slow.

Dijkstra (1956)
 $O(m + n \log n)$, but only for non-negative weights.

Algorithm	Weights	Running time
Bellman-Ford [Bellman '58], [Ford '56], [Moore '59], [Shimbel '55]	Reals	$O(mn)$
Dijkstra's + Fib heap [Fredman, Tarjan '87]	Nonnegative Reals	$O(m + n \log n)$
This work	Reals	$\tilde{O}(mn^{8/9})$

PROPOSED SOLUTION

Novelty: Introduces a randomized approach to eliminate negative edges gradually, using price functions.

Core Idea:

- Reweight edges using price functions.
- Isolate and eliminate negative vertices step-by-step.
- Apply Dijkstra's algorithm once negative edges are removed.

ALGORITHM OVERVIEW

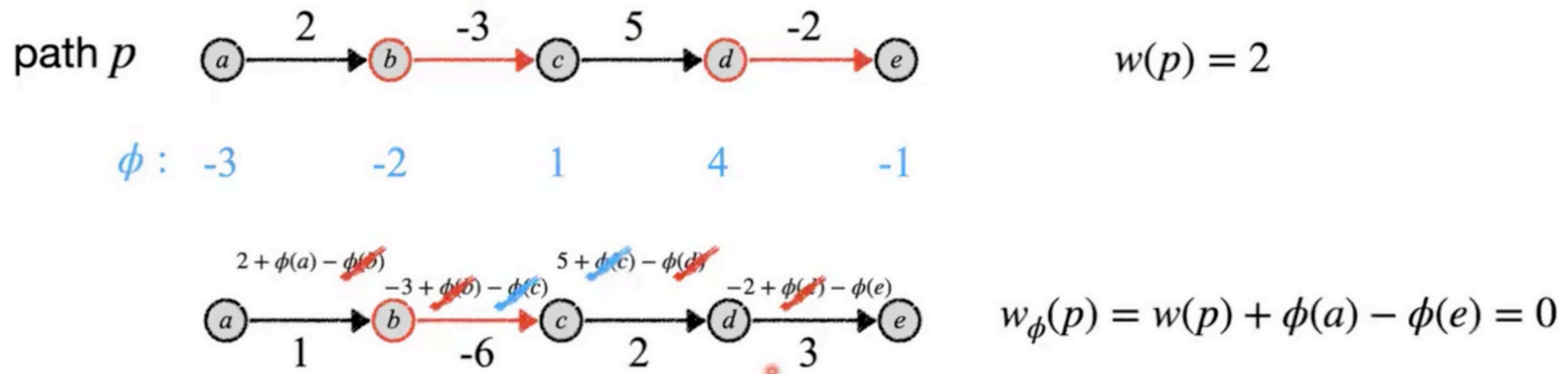
- Betweenness Reduction (adjust weights to minimize negative influence).
- Find Negative Sandwich / Independent Set (isolate problematic edges).
- Reweighting (make negative edges "remote").
- Hop-Reduction (eliminate remaining negatives).



PRICE FUNCTIONS

Goal: Modify edge weights without changing the shortest path.

- Use a price function $\varphi: V \rightarrow \mathbb{R}$
- Define new weights $w(u,v) = w(u,v) + \varphi(u) - \varphi(v)$



GRADUALLY ELIMINATE NEGATIVE EDGES

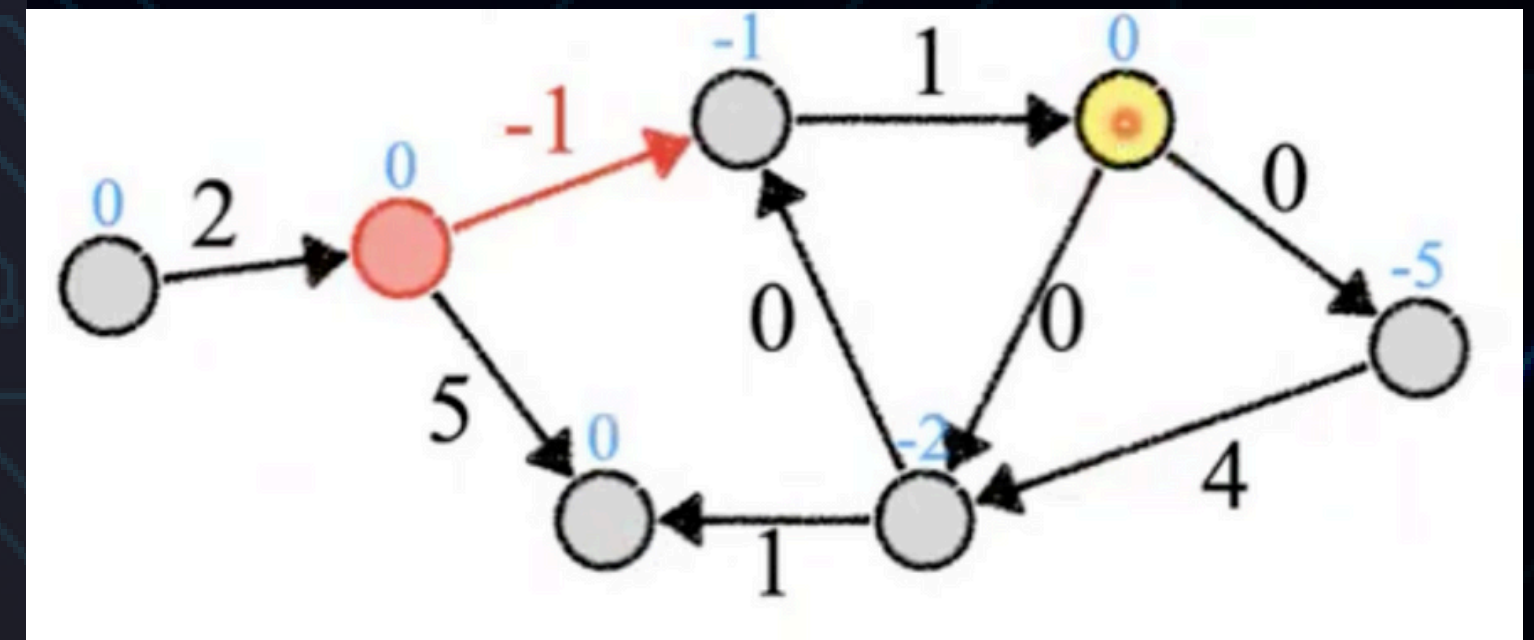
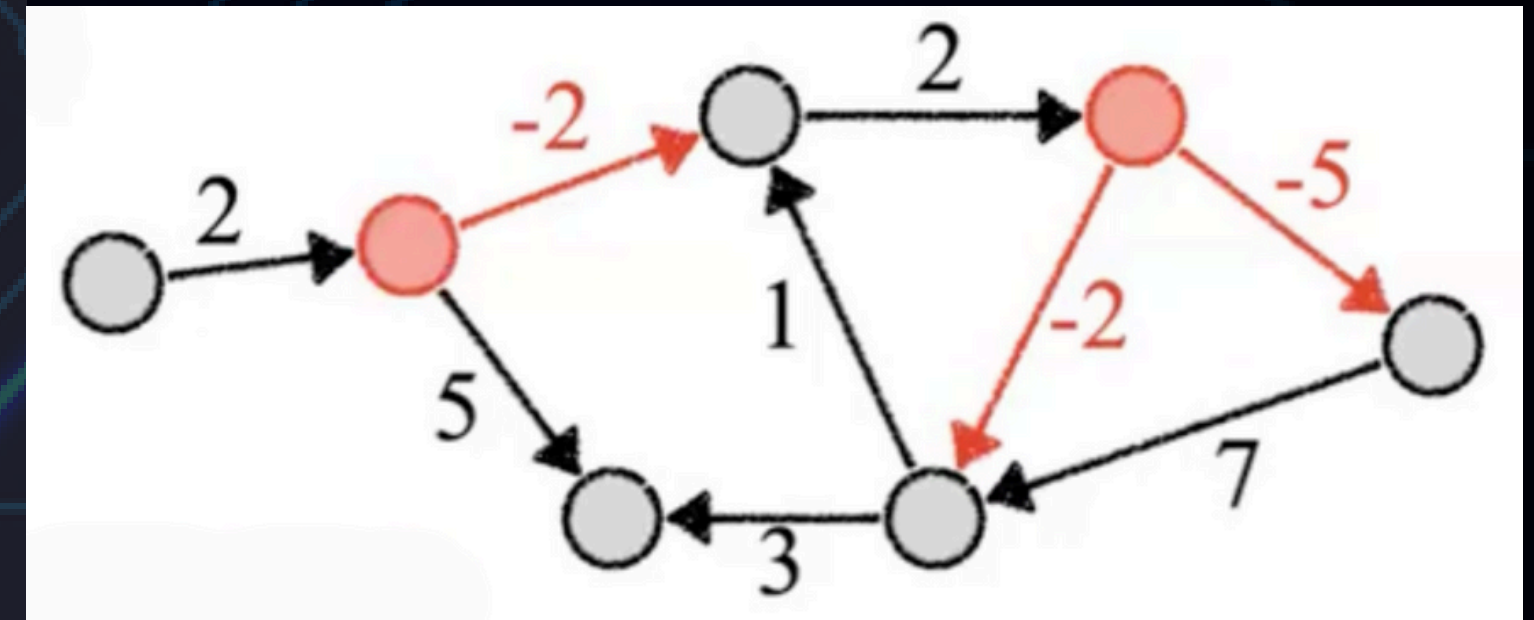
Start with graph with some weight function.

Negative edges: those with negative weight initially.

Negative vertices: vertices with outgoing negative edges.

A price function eliminates negative vertex u if $w(u, v) \geq 0$ for all $(u, v) \in E$

Strategy: Gradually eliminate negative vertices.
Initially non-negative edge weights remain non-negative.



FIND NEGATIVE SANDWICH OR INDEPENDENT SET

- Searches for either a "negative sandwich" (clustered negative edges) or a "1-hop independent set" (negative vertices without direct negative connections)
- If an independent set is found, negative edges from these vertices can be eliminated directly using a price function.
- If a negative sandwich is found, the algorithm proceeds to next step.

REWEIGHTING (MAKING EDGES REMOTE) & HOP-REDUCTION

- Applies a second price function (φ_2) to the graph
- The goal is to make the negative edges within a found negative sandwich "r-remote", meaning they are positioned such that they affect a limited part of the graph.
- Employs hop-limited shortest paths, restricting the number of negative edges allowed in a path.
- This technique simplifies the problem of computing shortest paths in the presence of the remaining negative edges.

After these steps, the graph is transformed, allowing the use of Dijkstra's to compute shortest paths. The final distances are adjusted based on the applied price functions. The algorithm can also detect negative-weight cycles.

COMPLEXITY ANALYSIS

Time Complexity:

- The overall time complexity of the algorithm is $\tilde{O}(mn^8/9)$.
- The improvements come from reducing the number of negative edges and efficiently applying the reweighting and hop-limited steps.

Space Complexity:

- The space complexity is $O(m + n)$, where m is the number of edges and n is the number of vertices.
- The space is used for storing the graph, price functions, and auxiliary data structures for hop-limited paths.

EXPERIMENTAL SETUP

Data:

- Random graphs generated for varying sizes. Different number of vertices

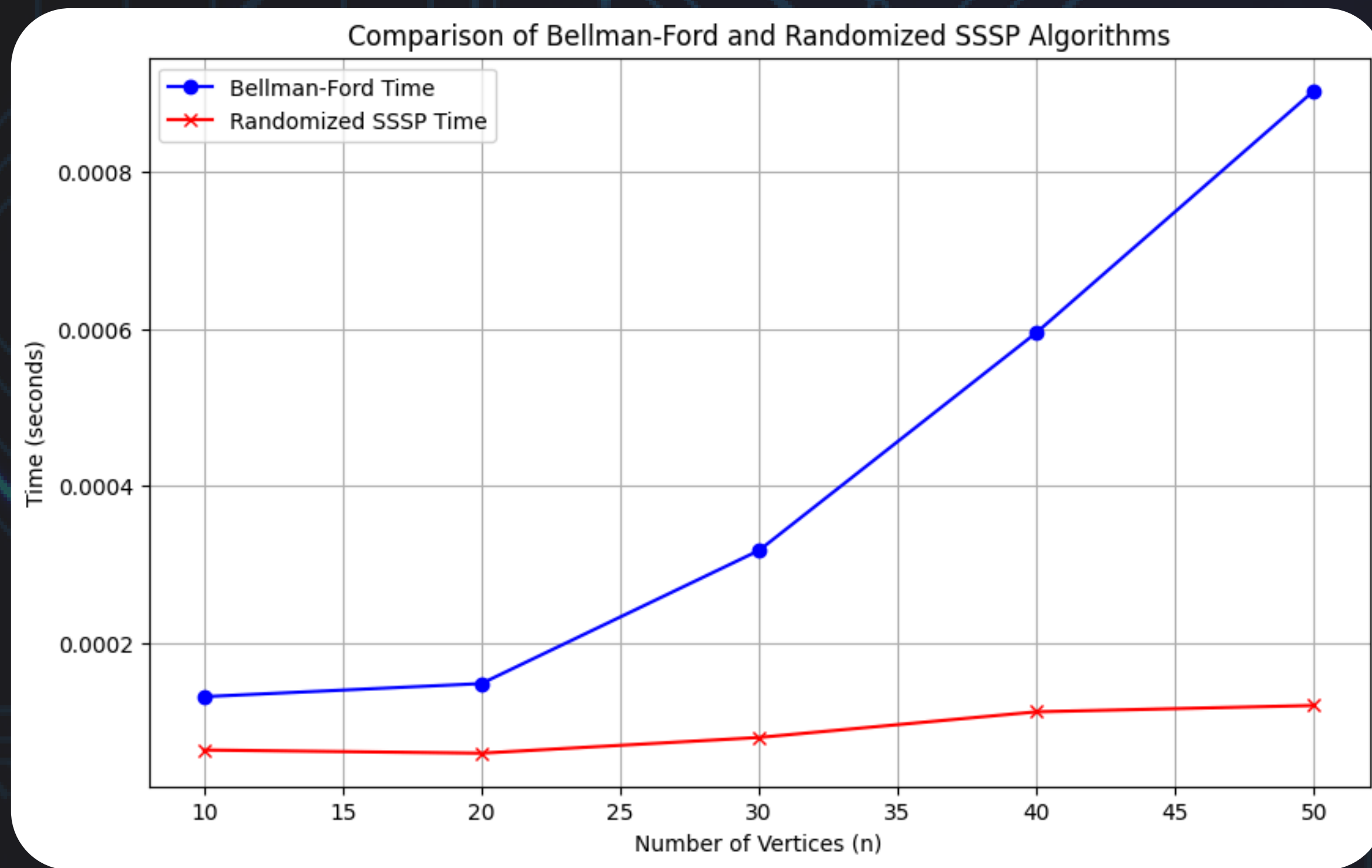
Implementation Details:

- Algorithms implemented in Python.
- Bellman-Ford and Randomized SSSP compared on random graphs.

Evaluation Metrics:

- Runtime: Time taken for each algorithm to compute the shortest path.

RESULTS



LIMITATIONS / FUTURE DIRECTIONS

Limitations:

- High Density of Negative Edges: Overhead in dense graphs.
- Large Graphs: Potential inefficiency with very large graphs.

Future Work:

- **Full Implementation Optimization:** Improve components like price function reweighting and $\Theta(k^{1/3}$ -elimination to achieve theoretical time complexity.

CONCLUSION

- The Randomized SSSP algorithm offers a faster solution for single-source shortest path problems on graphs with negative real-weighted edges.
- The algorithm reduces the time complexity from $O(mn)$ to $\tilde{O}(mn^{8/9})$.
- Provides a significant speedup in graphs with negative edges, making it a strong choice for large-scale real-world problems like currency arbitrage and network optimization.

REFERENCES

- Fineman, J.T. (2024), "Single-Source Shortest Paths with Negative Real Weights in $O(mn^{8/9})$ Time," Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC 2024), pp. 3–14. [DOI: 10.1145/3618260.3649614](https://doi.org/10.1145/3618260.3649614)